

中山大学网络空间安全学院必修课程

第5节 循环结构

C Programming

岳 晟

网络空间安全学院 工学园2栋508

yuesh5@mail.sysu.edu.cn

2026年秋季

结构化程序设计 (Structured Programming Theorem)

- **任何**复杂的程序逻辑，都可以用**三种基本结构**（程序流）组合而成：
 - 1. 顺序结构**：程序按照代码书写的先后顺序，自上而下逐条执行
 - 2. 选择结构**：根据条件判断的结果，选择执行不同的代码路径
 - 3. 循环结构**：在满足条件的情况下，重复执行一段代码

实例：计算平均分

```
#include <stdio.h>

int main(void) {
    double s1, s2, s3, avg;

    scanf("%lf%lf%lf", &s1, &s2, &s3);

    avg = (s1 + s2 + s3) / 5.0;
    printf("average = %.2f\n", avg);

    return 0;
}
```

人数一旦改变，程序就需要修改

```
#include <stdio.h>

int main(void) {
    int n, i = 1;
    double score, sum = 0, avg;
    scanf("%d", &n);

    while (i <= n) {
        scanf("%lf", &score);
        sum += score;
        i++;
    }

    avg = sum / n;
    printf("average = %.2f\n", avg);

    return 0;
}
```

主要内容

1. while 循环
2. for 循环
3. do while 循环
4. break 与 continue



while 循环

while(表达式)
语句

if(表达式)
语句

while 循环

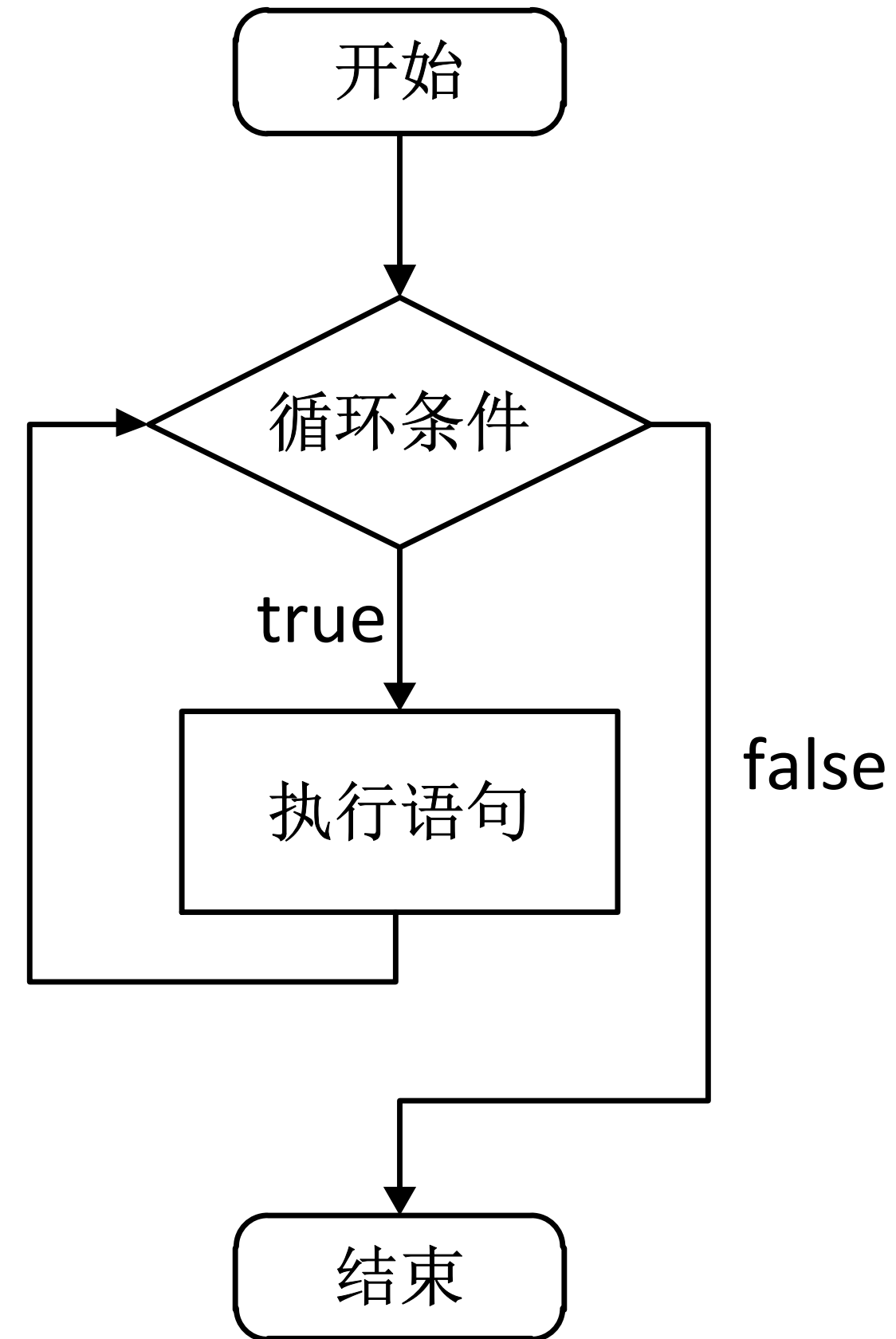
```
while(表达式)
{
    复合语句
}
```

```
while(表达式)
    简单语句
```

- 在while循环中，首先判断**循环条件表达式**是否为**真**，如果为真，就执行后面的语句/语句块（**循环体**）
- 之后不断**重复**上述过程，只要循环条件表达式**为真就执行**循环体
- 直到判断**循环条件表达式为假**，不再执行循环体，离开循环结构

while 循环

while(表达式)
语句



练习

□ $1 + 2 + 3 + \dots + n = ?$

```
#include <stdio.h>

int main(void) {
    int n, i = 1, sum = 0;
    scanf("%d", &n);

    while (i < n) {
        sum += i;
        i++;
    }

    printf("%d\n", sum);
    return 0;
}
```

程序对不对?

问题

□ 下述两段代码结束后，变量 i 和 sum 的值分别是否一样？

```
int i=1, sum=0;

while (i <= 6) {
    sum += i;
    i++;
}
```

```
int i=0, sum=0;

while (i < 6) {
    i++;
    sum += i;
}
```

死循环

□ $1 + 2 + 3 + \dots + 100 = ?$

```
#include <stdio.h>
int main(void) {
    int n, i=1, sum=0;
    scanf("%d", &n);

    while (i < n) {
        sum += i;
    }

    printf("%d\n", sum);
    return 0;
}
```

```
#include <stdio.h>
int main(void) {
    int n, i=1, sum=0;
    scanf("%d", &n);

    while (i < n)
        sum += i;
        i++;

    printf("%d\n", sum);
    return 0;
}
```

□ **死循环**：循环终止条件永远无法满足

练习

□ 从键盘输入一行文本，统计并输出该行中包含的字符个数

```
#include <stdio.h>

int main(void){
    int n=0;

    printf("Input a string:");
    while(getchar()!='\n') n++;

    printf("Number of characters: %d\n", n);
    return 0;
}
```

猜数游戏



```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void) {
    int answer, guess;

    srand((unsigned)time(NULL));
    answer = rand() % 100 + 1; // 随机生成1-100间整数

    printf("请输入一个 1-100 之间的整数 : ");
    scanf("%d", &guess);

    if (guess < answer) {
        printf("Too small!\n");
    } else if (guess > answer) {
        printf("Too large!\n");
    } else {
        printf("Correct!\n");
    }

    return 0;
}
```

使用标志控制循环



```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void) {
    int answer, guess;
    srand((unsigned)time(NULL));
    answer = rand() % 100 + 1;

    int found = 0;
    while (!found) {
        printf("请输入你的猜测 : ");
        scanf("%d", &guess);

        if (guess < answer) {
            printf("Too small!\n");
        } else if (guess > answer) {
            printf("Too large!\n");
        } else {
            printf("Correct!\n");
            found = 1;
        }
    }
}
```

不确定循环与计数循环

不确定循环：计算字符数

```
/*while count string*/
#include <stdio.h>

int main(void){
    int n=0;

    printf("Input a string:");
    while(getchar()!='\n') n++;

    printf("Number of chars: %d\n", n);
    return 0;
}
```

计术循环：1-100求和

```
/*loop*/
#include<stdio.h>

int main(void) {
    int sum = 0;
    int i = 1;
    while (i < 100)
    {
        sum += i++;
    }
    printf("The sum is %d\n",sum);
}
```

for 循环

for(表达式1; 表达式2; 表达式3)
语句

表达式1：初始化

只执行一次，可为多个变量设置初值

表达式2：测试条件

根据表达式真假决定是否继续执行循环

表达式3：修改循环变量

使循环变量增加，在循环结束时求值

```
#include <stdio.h>

int main(void) {

    int n, i, sum = 0;

    scanf("%d", &n);

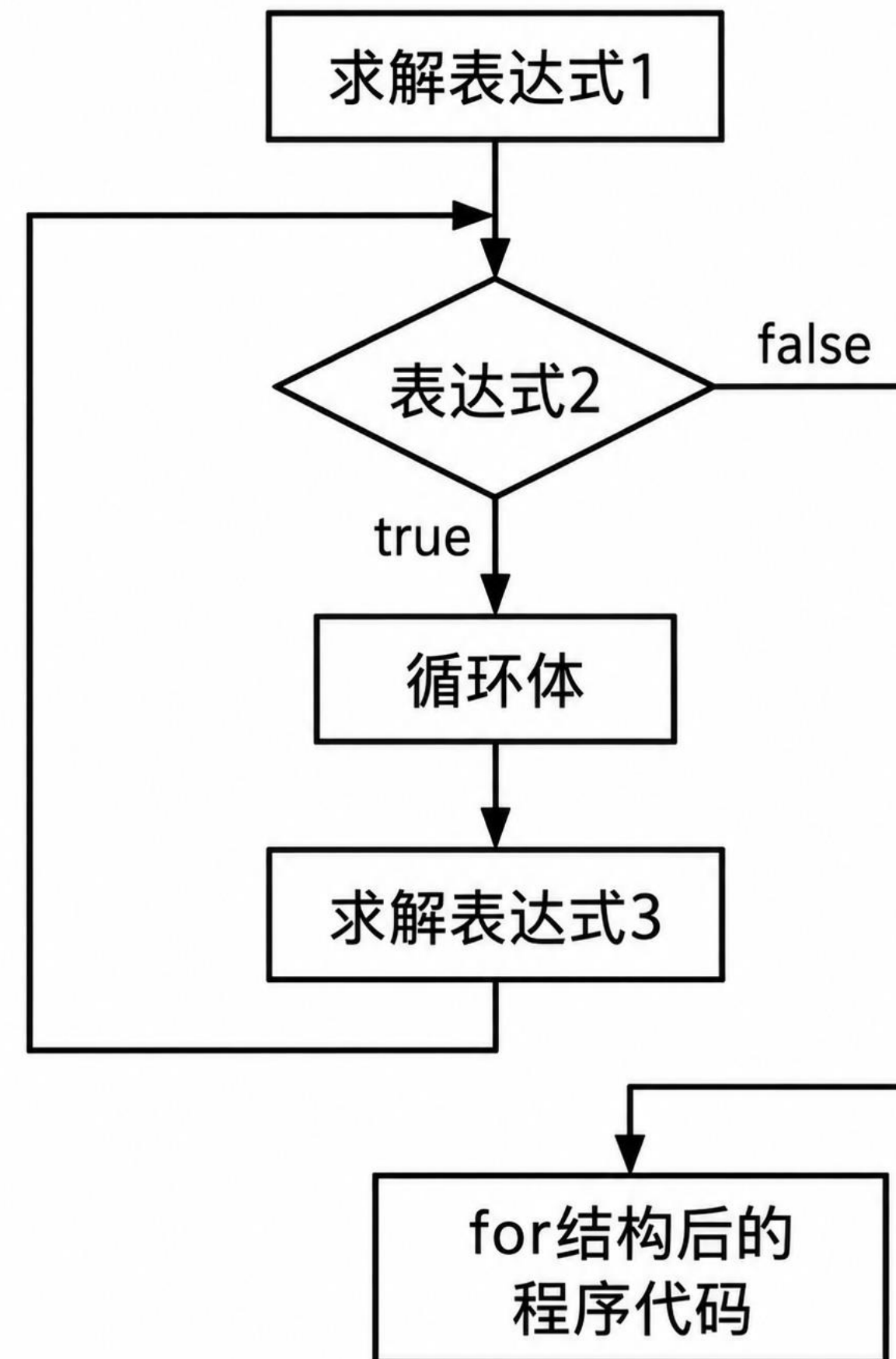
    for (i=1; i<=n; i++) {
        sum += i;
    }

    printf("%d\n", sum);

    return 0;
}
```

for 循环

for(表达式1; 表达式2; 表达式3)
语句



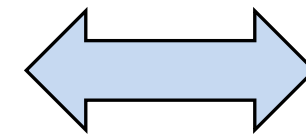
for 循环的最常见形式

```
for(循环变量赋初值; 循环条件; 循环变量增值)
{
    语句块
}
```

表达式3要使循环变量向**趋于循环“结束”的方向**进行，避免死循环

for 循环与 while 循环的关系

```
int sum = 0;
for (int i = 1; i <= 10; i++) {
    sum += i;
}
```



```
int i = 1; sum = 0;
while (i <= 10) {
    sum += i;
    i++;
}
```

既然 for 循环等价于 while 循环，那为什么需要 for 语句呢？

for 循环

```
int i = 1, sum = 0;  
while (i <= 10) {  
    sum += i;  
}
```

- ① 现实中有大量问题需要**枚举**有明确开始、结束数字或事物的序列
- ② 使用 for 语句符合自然语言语义，语法简明 (开始、结束、增量)
- ③ 易于调试，不易出现死循环

循环变量定义位置

```
#include <stdio.h>

int main() {
    int sum = 0;
    int i;
    for (i = 1; i < 100; i++) {
        sum += i;
    }
    printf("%d %d", i, sum);
}
```

```
#include <stdio.h>

int main() {
    int sum = 0;
    for (int i = 1; i < 100; i++) {
        sum += i;
    }
    printf("%d %d", i, sum);
}
```

for 循环内定义的变量只有块作用域 (仅限 for 循环内部)

表达式省略

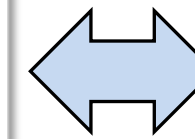
□ for 中的三个表达式都可以省略，但**两个分号不能省略**

```
int sum = 0;
for (int i=1; i<=10;){
    sum += i;
    i++;
}
```

省略表达式3

```
int sum = 0, i = 1;
for (;i<=10;) {
    sum += i;
    i++;
}
```

省略表达式1



```
int i = 1, sum = 0;
while (i<=10) {
    sum += i;
    i++;
}
```

等价的while写法

省略表达式2

```
int sum = 0;  
for (int i=1;;i++) {  
    sum += i;  
}
```



```
int i = 1, sum = 0;  
while (1) {  
    sum += i;  
    i++;  
}
```

for 循环

3个表达式可以同时省略

```
for(;;)
```

```
while(1)
```

for 循环

表达式1 和 表达式3 可以是一个简单表达式也可以是逗号表达式

```
for (sum=0, i=1; i<=100; i++) sum += i;
```

```
for (i=1, j=10, sum=0; i<=10; i++, j--) sum += i*j;
```

练习

□ 将以下程序改写成 for 循环

```
#include <stdio.h>

int main(void){
    int n = 0;

    while(getchar()!='\n') n++;

    printf("%d\n", n);

    return 0;
}
```

```
#include <stdio.h>

int main(void){
    int n;

    for (n=0; getchar()!='\n'; n++);

    printf("%d\n", n);

    return 0;
}
```

循环嵌套

□ 一个循环体内又包含另一个完整的循环结构，称为**循环的嵌套**

```
while(表达式1)
{
    while(表达式2)
    {...}
}
```

```
for(表达式1.1; 表达式1.2; 表达式1.3)
{
    for(表达式2.1; 表达式2.2; 表达式2.3)
    {...}
}
```

循环嵌套

```
for (int i=1; i<=3; i++) {  
    for (int j=1; j<=3; j++) {  
        printf("%d*%d=%d\t", i, j, i * j);  
    }  
    printf("\n");  
}
```

循环嵌套

外循环



```
for (int i=1; i<=3; i++) {  
    for (int j=1; j<=3; j++) {  
        printf("%d*%d=%d\t", i, j, i * j);  
    }  
    printf("\n");  
}
```

循环嵌套

内循环 →

```
for (int i=1; i<=3; i++) {  
    for (int j=1; j<=3; j++) {  
        printf("%d*%d=%d\t", i, j, i * j);  
    }  
    printf("\n");  
}
```

内循环结束才会开始下一轮外循环

练习

□ 利用嵌套循环输出一个 4×4 的整数矩阵

1	2	3	4
2	4	6	8
3	6	9	12
4	8	12	16

```
#include <stdio.h>

int main(void) {

    for (int i=1; i<=4; i++) {
        for (int j=1; j<=4; j++) {
            printf("%-4d", i*j);
        }
        printf("\n");
    }
    return 0;
}
```

练习

□ 输出九九乘法表

1*1=1								
2*1=2	2*2=4							
3*1=3	3*2=6	3*3=9						
4*1=4	4*2=8	4*3=12	4*4=16					
5*1=5	5*2=10	5*3=15	5*4=20	5*5=25				
6*1=6	6*2=12	6*3=18	6*4=24	6*5=30	6*6=36			
7*1=7	7*2=14	7*3=21	7*4=28	7*5=35	7*6=42	7*7=49		
8*1=8	8*2=16	8*3=24	8*4=32	8*5=40	8*6=48	8*7=56	8*8=64	
9*1=9	9*2=18	9*3=27	9*4=36	9*5=45	9*6=54	9*7=63	9*8=72	9*9=81

break/continue 语句

```
break;
```

- ❑ **break** 语句可以使当前流程跳出switch结构，继续执行 switch 结构后的下一条语句
- ❑ **break** 语句还用来从循环体中跳出整个循环结构，即提前结束循环，执行循环结构后的下一条语句

```
continue;
```

- ❑ **continue** 语句可以提前结束本次循环，而接着执行下次循环

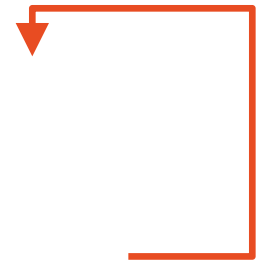
break/continue 语句

```
for (int i=1; i<=10; i++)  
{  
    if (i == 5) break;  
    printf("i = %d\n",i);  
}  
return 0;
```



当 $i = 5$ ，满足 if 语句的判断，执行 break 语句，跳出整个循环，循环结束

```
for (int i=1; i<=10; i++)  
{  
    if (i == 5) continue;  
    printf("i = %d\n", i);  
}  
return 0;
```



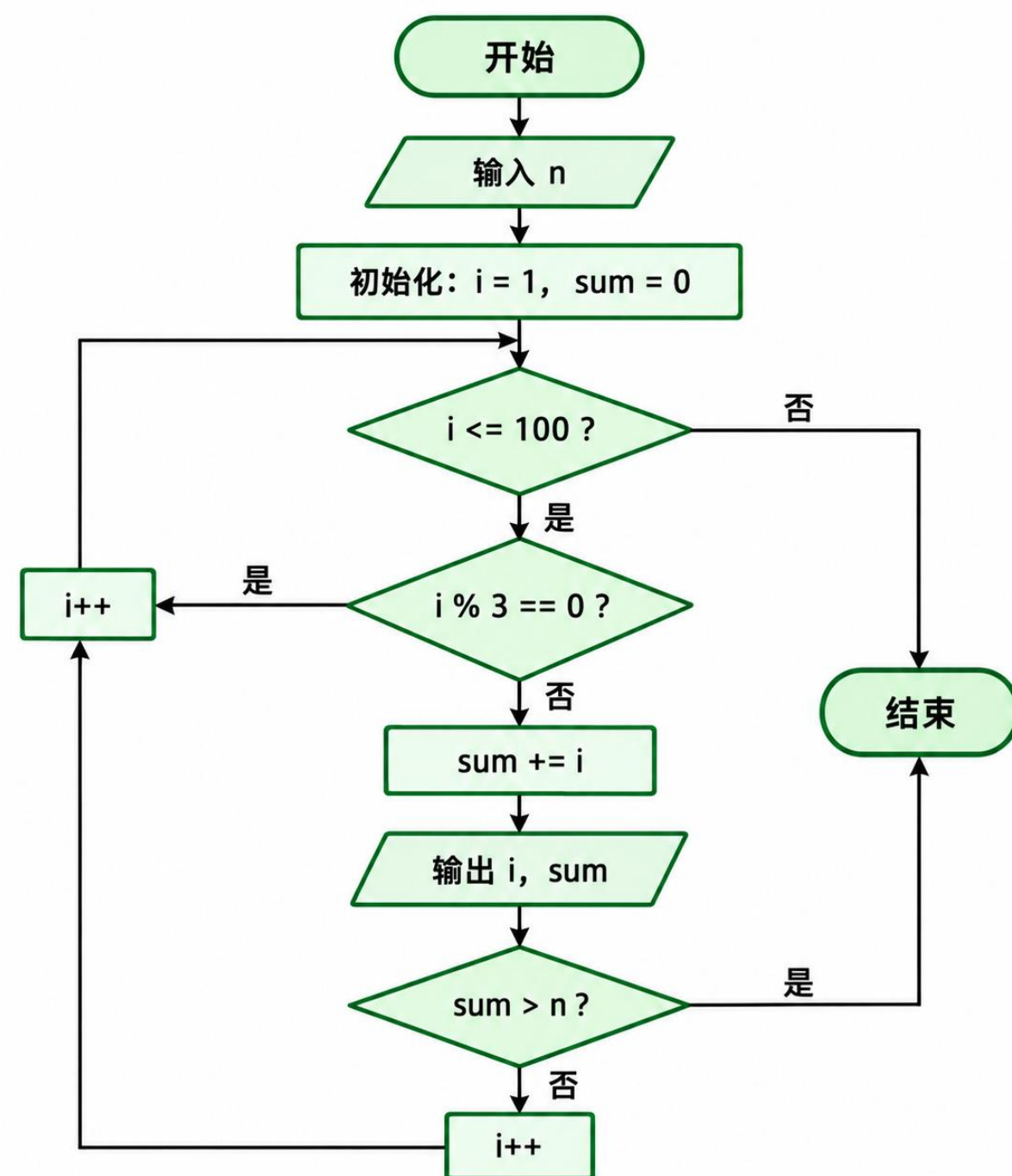
当 $i = 5$ ，满足 if 语句的判断，执行 continue 语句，跳过本次循环中 continue 语句后面语句的执行，转入下一次循环

练习

- 按顺序打印 1-100 中的整数，跳过 3 的倍数，当所有打印数字总和大于 n 时结束

```
#include <stdio.h>
int main(void) {
    int n;
    scanf("%d", &n);

    for (int i=1, sum=0; i<=100; i++) {
        if (i % 3 == 0) continue;
        sum += i;
        printf("i=%-5d sum=%d\n", i, sum);
        if (sum > n) break;
    }
    return 0;
}
```

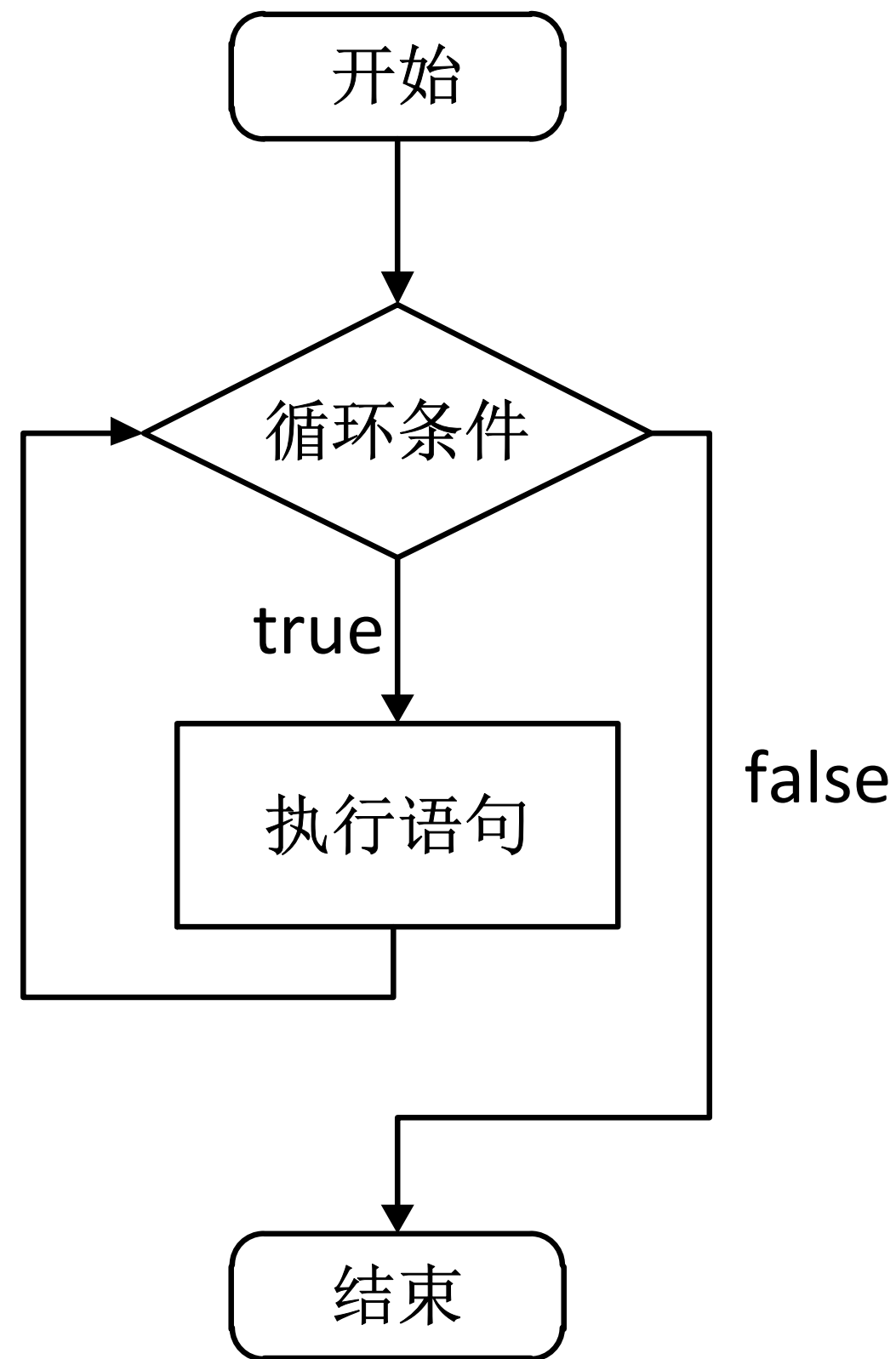


do while 循环

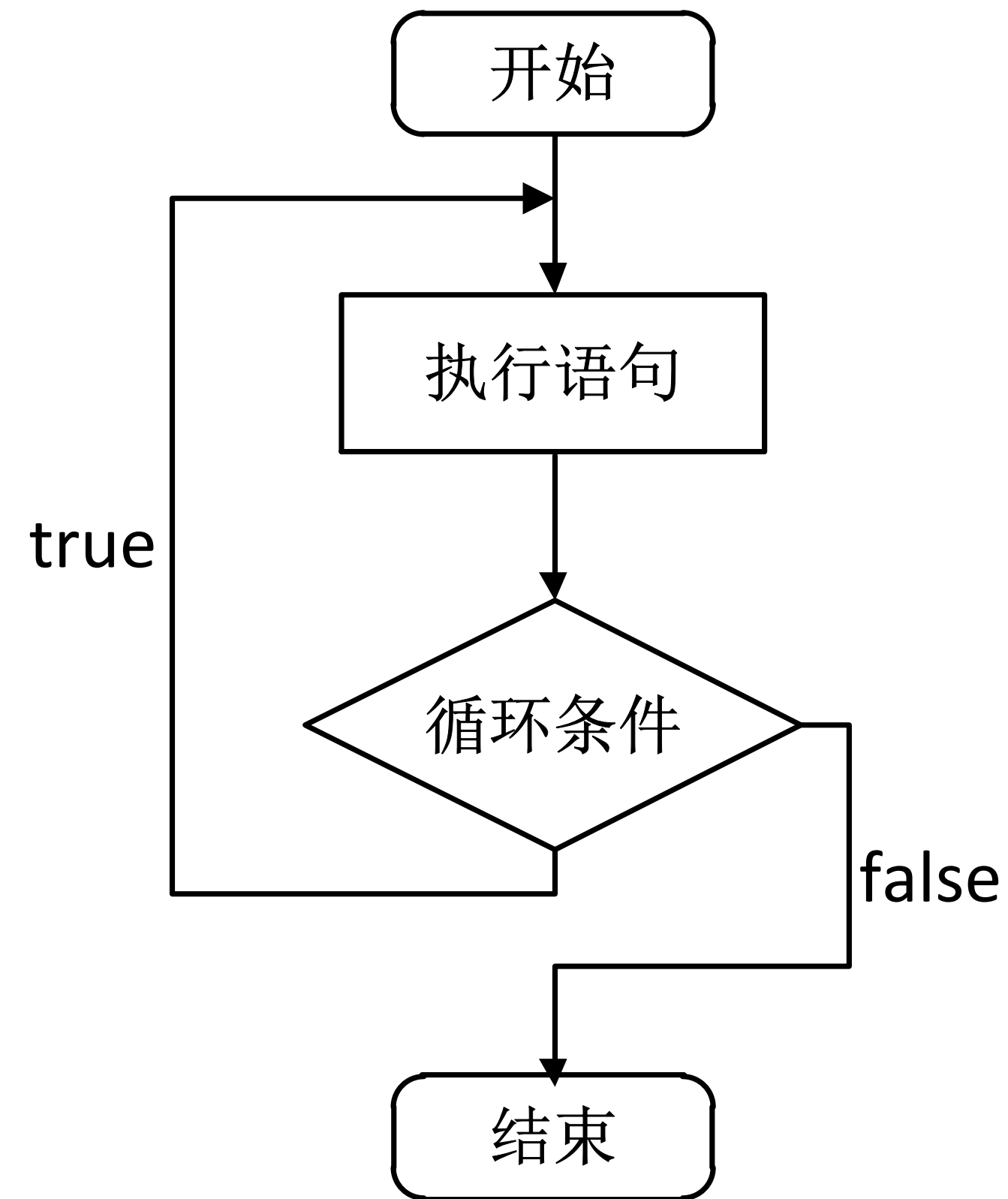
```
do  
{  
    复合语句  
}  
while(表达式);
```

- 先执行一次循环体语句，然后判断表达式，当表达式的值为真时，返回重新执行循环体语句
- 如此反复，直到判断到表达式的值为假，结束循环

do while 循环



while



do while (至少执行一次)

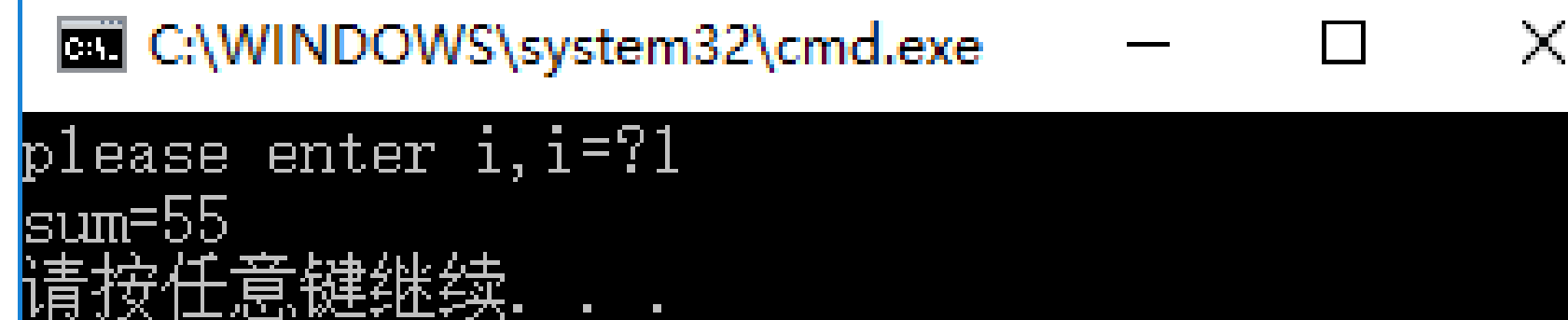
do while 循环

□ 当输入为 **1**，以下程序分别输出？

```
#include <stdio.h>

int main() {
    int i, sum = 0;
    scanf("%d", &i);

    while (i <= 10) {
        sum += i;
        i++;
    }
```

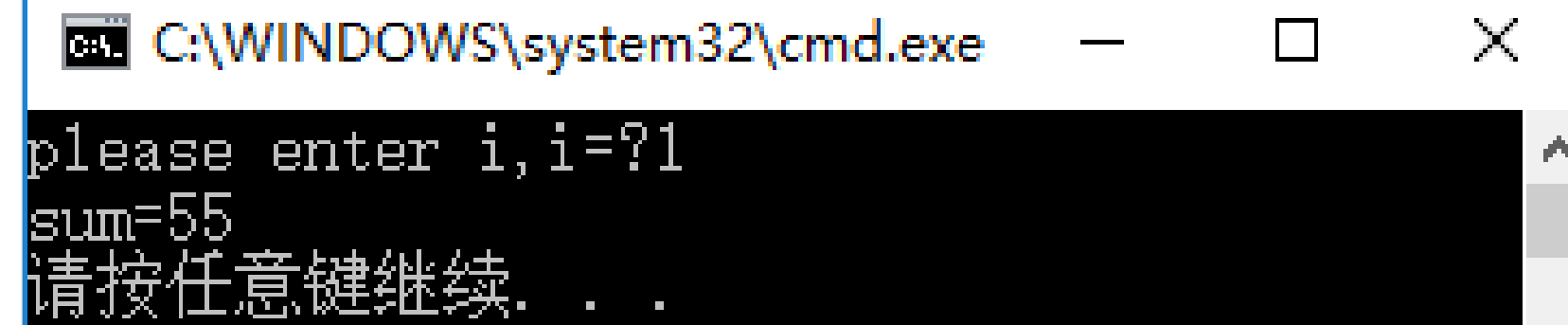


```
C:\WINDOWS\system32\cmd.exe
please enter i,i=?
1
sum=55
请按任意键继续. . .
```

```
#include <stdio.h>

int main() {
    int i, sum = 0;
    scanf("%d", &i);

    do {
        sum += i;
        i++;
    } while (i<=10);
```



```
C:\WINDOWS\system32\cmd.exe
please enter i,i=?
1
sum=55
请按任意键继续. . .
```

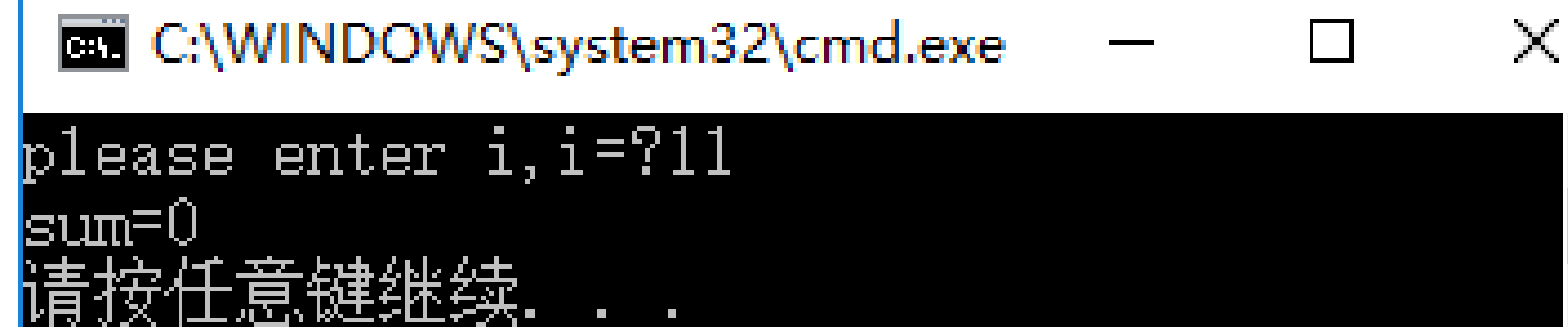
do while 循环

□ 当输入为 **11**，以下程序分别输出？

```
#include <stdio.h>

int main() {
    int i, sum = 0;
    scanf("%d", &i);

    while (i <= 10) {
        sum += i;
        i++;
    }
```

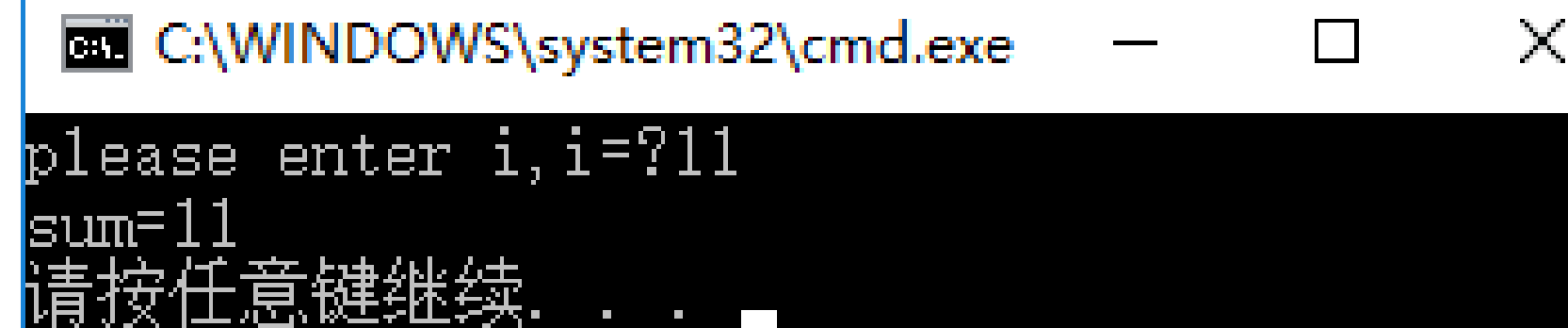


```
C:\WINDOWS\system32\cmd.exe
please enter i,i=?11
sum=0
请按任意键继续. . .
```

```
#include <stdio.h>

int main() {
    int i, sum = 0;
    scanf("%d", &i);

    do {
        sum += i;
        i++;
    } while (i<=10);
```



```
C:\WINDOWS\system32\cmd.exe
please enter i,i=?11
sum=11
请按任意键继续. . .
```

思考

以下程序会输出什么？

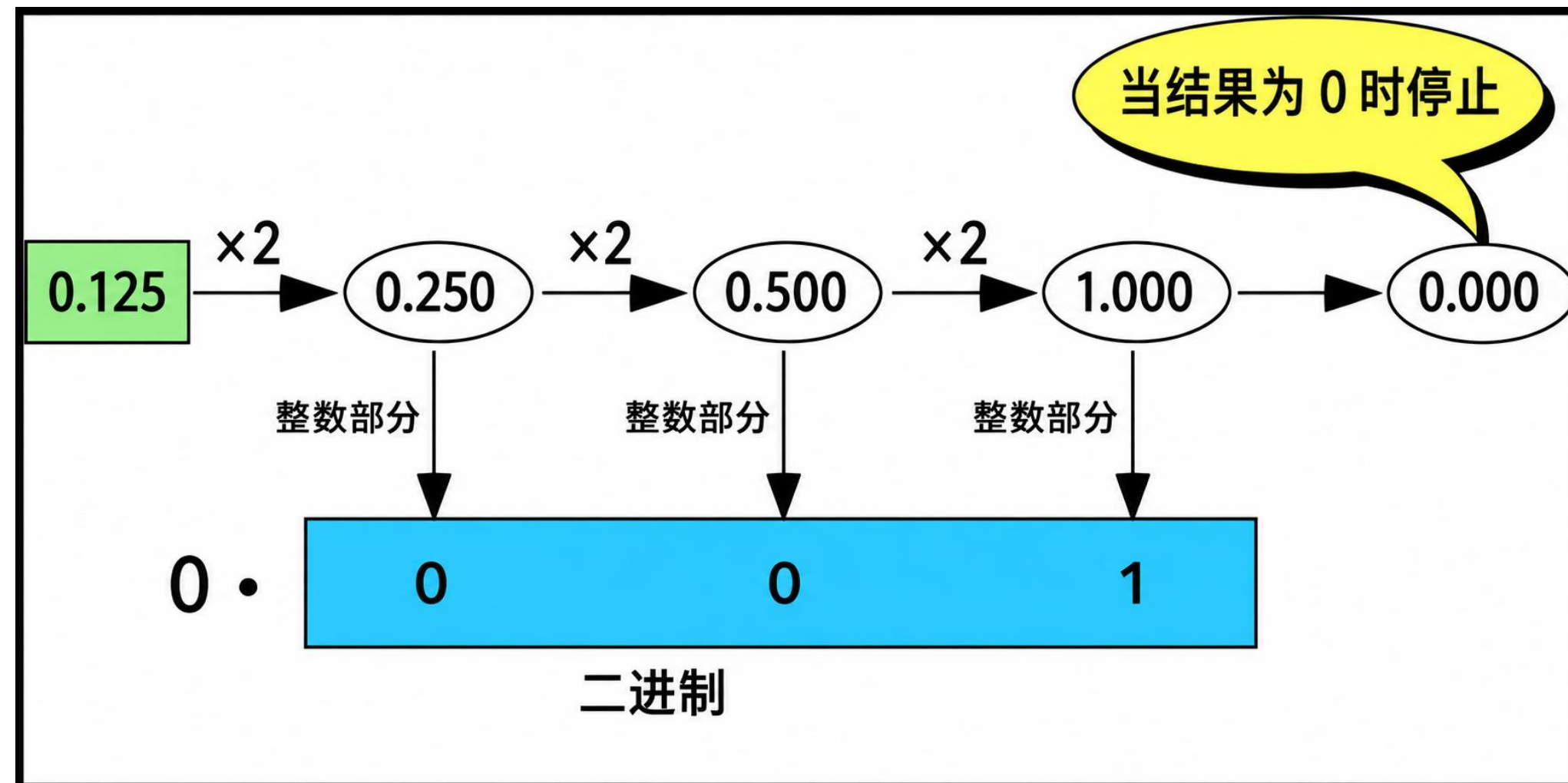
```
#include <stdio.h>

int main(void) {
    double x = 0.0;
    while (x != 1.0) {
        x += 0.1;
    }
    printf("x=%lf\n", x);

    return 0;
}
```

实数比较

- ❑ 计算机中 float/double 类型浮点数大多是实际数的近似值
- ❑ 计算机中小数的二进制表示



0.15
x2 0.30 -- 0
x2 0.60 -- 0
x2 1.20 -- 1 *
x2 0.40 -- 0 *
x2 0.80 -- 0 *
x2 1.60 -- 1 *
x2 1.20 -- 1 ...

$0.15 = (0.0010011001...)_{2}$

实数比较

- 两个实数变量**不能**使用 `==` 或 `!=` 直接**比较**，或**判断实数变量为零**
 - 特别是循环语句若使用类似 `x==y` 这样的比较表达式，可能导致死循环或循环中止
- 实际中，**实数比较运算**一般仅使用 `>` 和 `<` 两个比较运算符
 - 判断实数**相等**使用 `fabs(x-y) < eps`
 - 判断实数**等于零**使用 `fabs(x) < eps`
 - `eps` 称为精度控制量，为合适的小数 (如 `1e-7`)

练习

□ 用以下公式计算 π ，直到某一项的绝对值小于 10^{-6} 为止 (该项不累加)

$$\pi = 4 * \left(1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \dots + \frac{1}{4n-3} - \frac{1}{4n-1} + \dots \right)$$

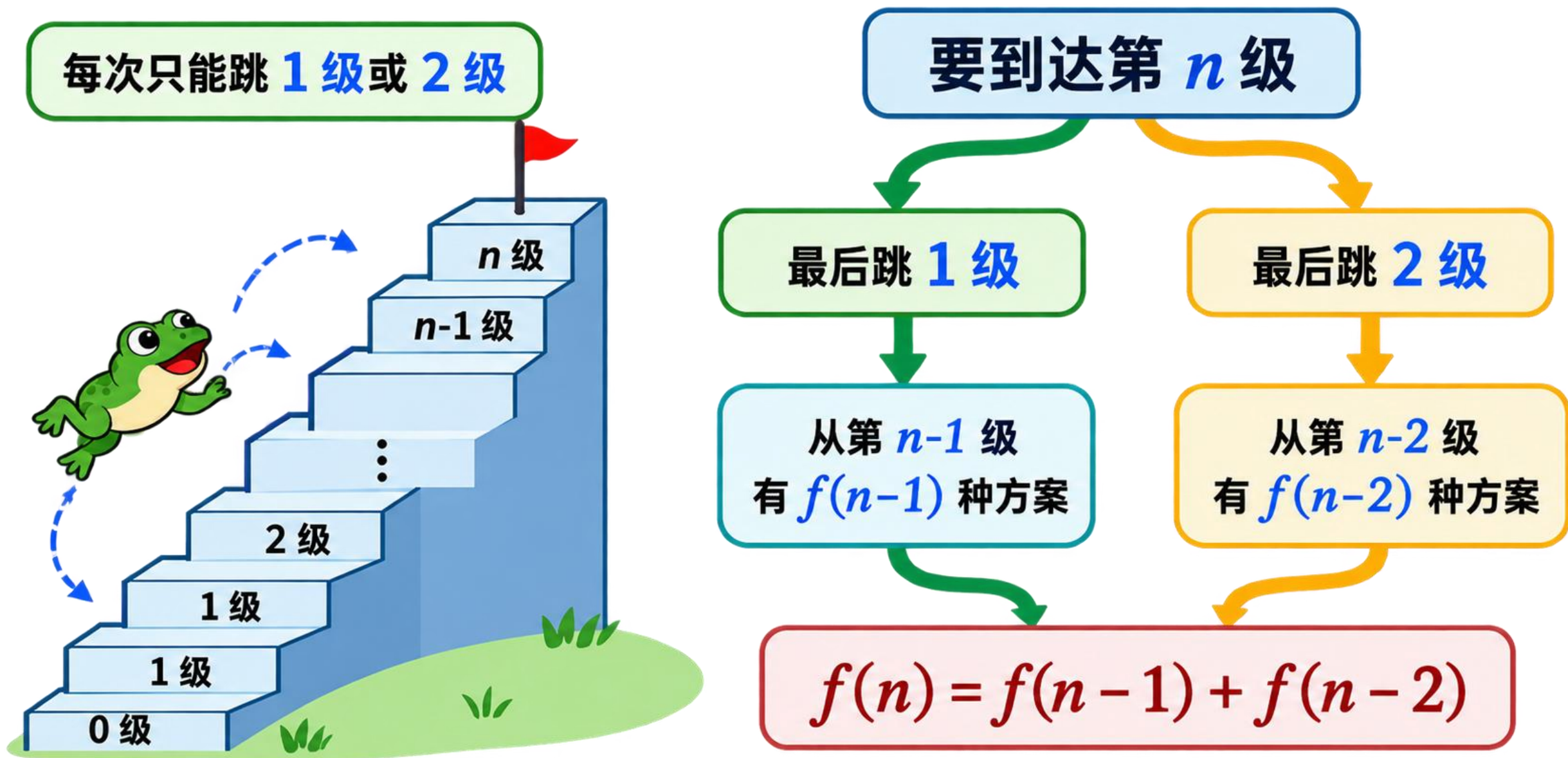
```
double pi = 0;
int i = 1;
const double eps = 1e-6;    // 精度保证5位
while (1.0/i > eps) {       // 不适用for循环
    pi += 1.0/i;
    i++;i++;
    pi -= 1.0/i;
    i++;i++;
}
pi *= 4;
printf("%.5f\n", pi);
```

练习

- 一个楼梯共有 n 级台阶，每次可以走 1 级或者走 2 级，问从第 0 级台阶走到第 n 级台阶一共有多少种方案？
- 例如，当 $n = 3$ 时，共有 3 种方案：
 1. $1 + 1 + 1$
 2. $1 + 2$
 3. $2 + 1$

练习

□ 一个楼梯共有 n 级台阶，每次可以走 1 级或者走 2 级，问从第 0 级台阶走到第 n 级台阶一共有多少种方案？



输入 n	输出
1	1
2	2
3	3
5	8
10	89

练习

- 一个楼梯共有 n 级台阶，每次可以走 1 级或者走 2 级，问从第 0 级台阶走到第 n 级台阶一共有多少种方案？

```
// f1 : 前一项，f2 : 当前项，f : 下一项
int f1=1, f2=2, f; if (n==1) f2=1;
for(int i=3; i<=n; i++){
    f = f1 + f2;
    f1 = f2;
    f2 = f;
}
printf("%d\n", f2);
```

输入 n	输出
1	1
2	2
3	3
5	8
10	89

练习

□ 输入一个大于 3 的整数 n ，判定它是否为素数

输入	输出
3	Yes!
4	No!
17	Yes!
25	No!
49	No!
97	Yes!

```
int n;
scanf("%d", &n);

int k = sqrt(n);
int is_prime = 1;
for (int i=2; i<=k; i++) {
    if (n % i == 0) {
        is_prime = 0;
        break;
    }
}
if (is_prime) printf("Yes!");
else printf("No!");
```

回顾：算法

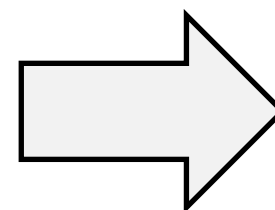
- ❑ **算法**：是计算机为解决一个问题而采取的方法和步骤
- ❑ 同一个问题可以有不同的算法，好的算法能极大提升**解决问题的效率**

例2 判断一个大于3的整数 n 是否为素数

逐个试除

从 2 试除到 $n - 1$
如果可以整除，则 n 不是素数

最多需 $n - 2$ 次计算



优化方法

从 2 试除到 $\sqrt{n}$
如果可以整除，则 n 不是素数

最多需 $\sqrt{n}$ 次计算

调试 (Debug) 与错误

- ❑ **调试 (Debug)**，又称除错，是发现和减少计算机程序错误的过程
- ❑ 程序错误的种类：
 - 语法 (编译/Compile) 错误，编译器会给出提示，如语句缺少 “;”
 - 运行时 (Runtime) 错误，程序运行时异常退出，给或不给错误信息
 - 逻辑错误，程序正常运行，但未得到期望的结果
- ❑ 调试是发现**后两类错误**的重要手段

调试方法

□ 调试方法一：

Where?

What?

- 使用 printf 函数在指定**位置**打印路径相关信息或**变量**值

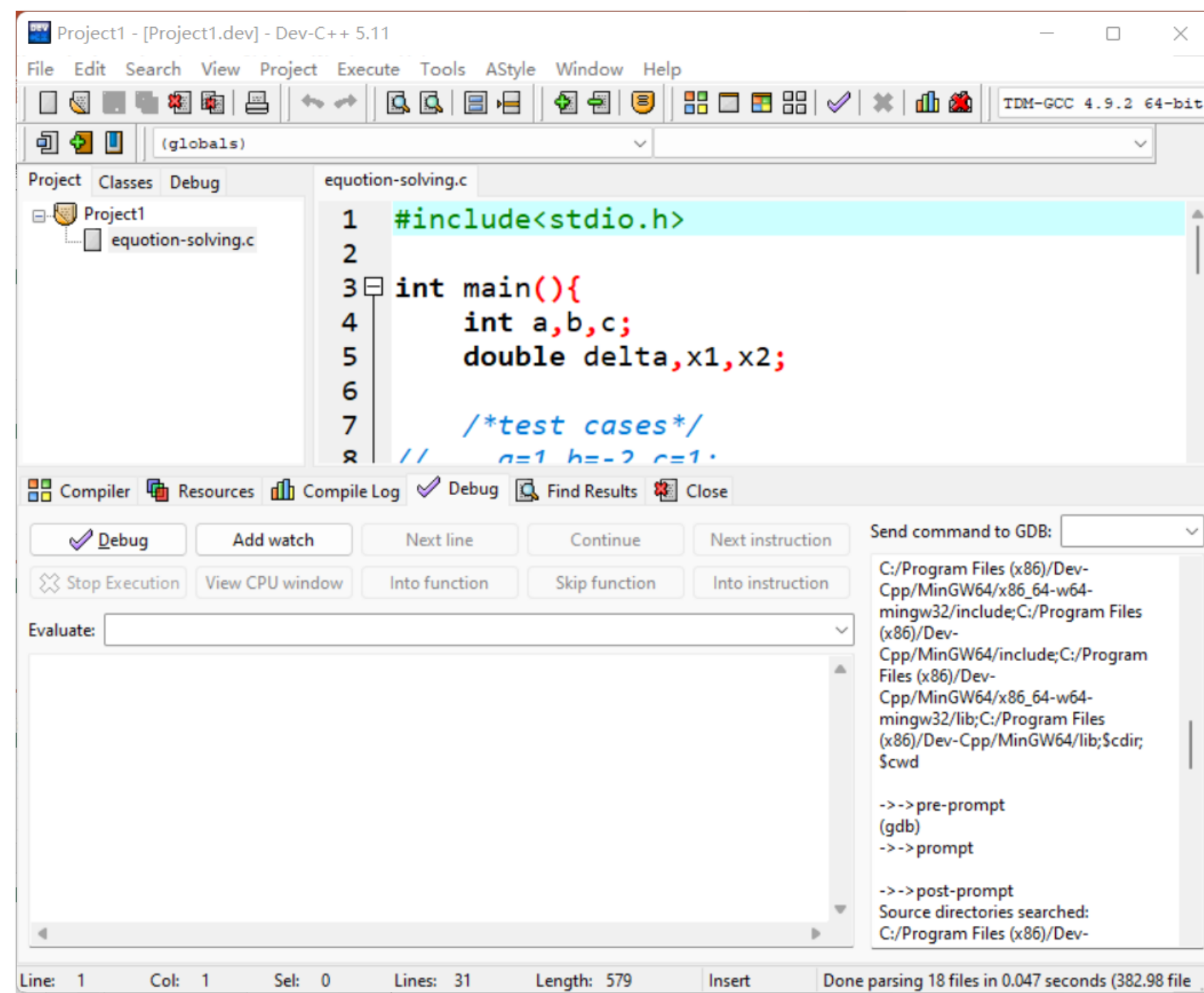
□ 调试方法二：

- 工具，如 **DEV-C++ GDB** (GNU Debugger)
- 语句跟踪，使用**断点 (Break Point)** 或执行下一句等指令跟踪程序执行路径
- 变量查看，在跟踪的过程中**观察变量的变化**

□ 因此需要掌握调试工具的使用以及调试的技巧

调试工具：DEV C++

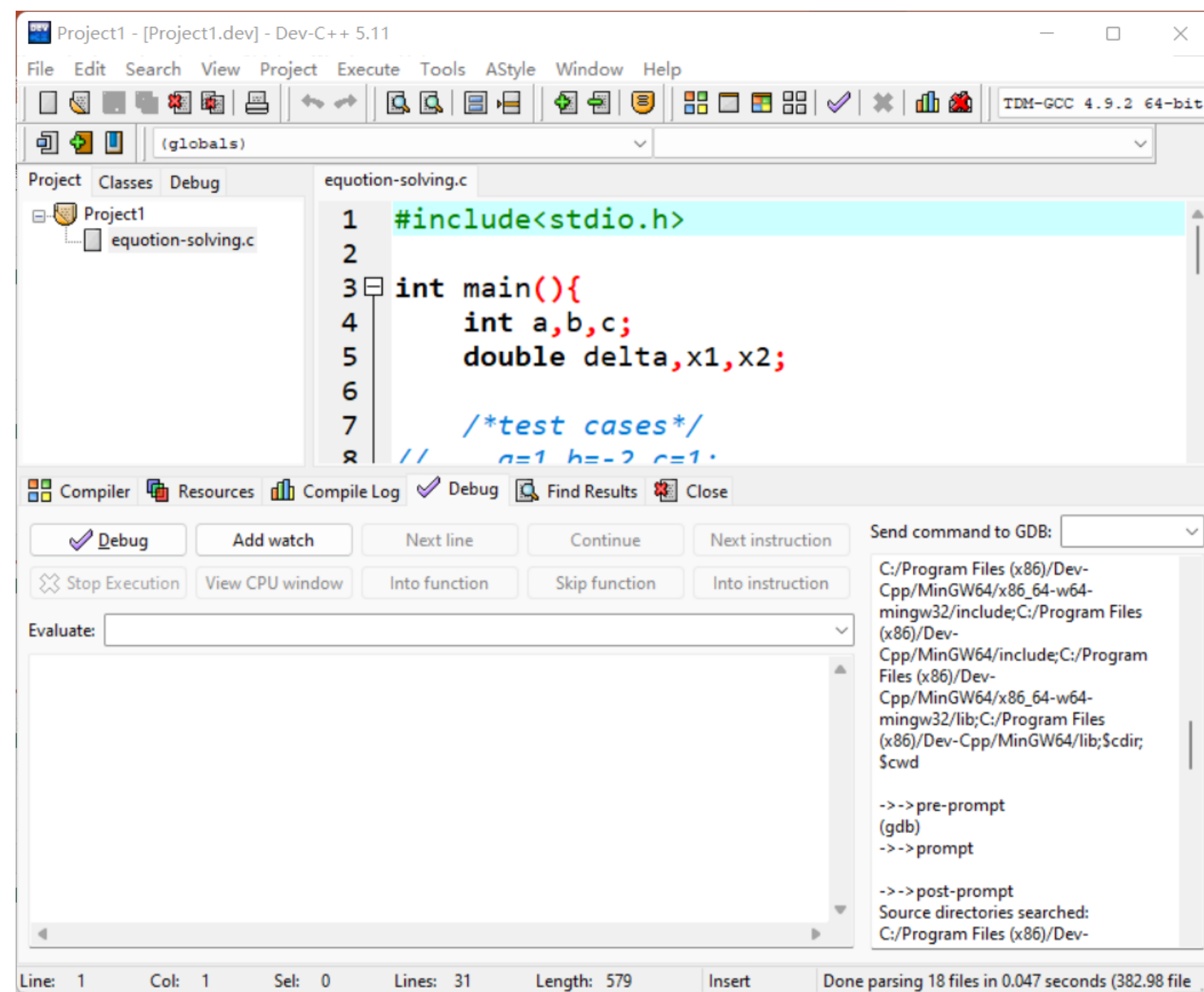
- 调试须知：必须先创建项目再调试
 - 创建一个项目，选择 控制台应用
 - 将要调试 C 代码添加到该项目
 - 点击代码，打开代码
 - 修改代码
 - 按 F12 编译通过
 - 按 F5 键开启调试



调试工具：DEV C++

□ 调试须知：必须先创建项目再调试

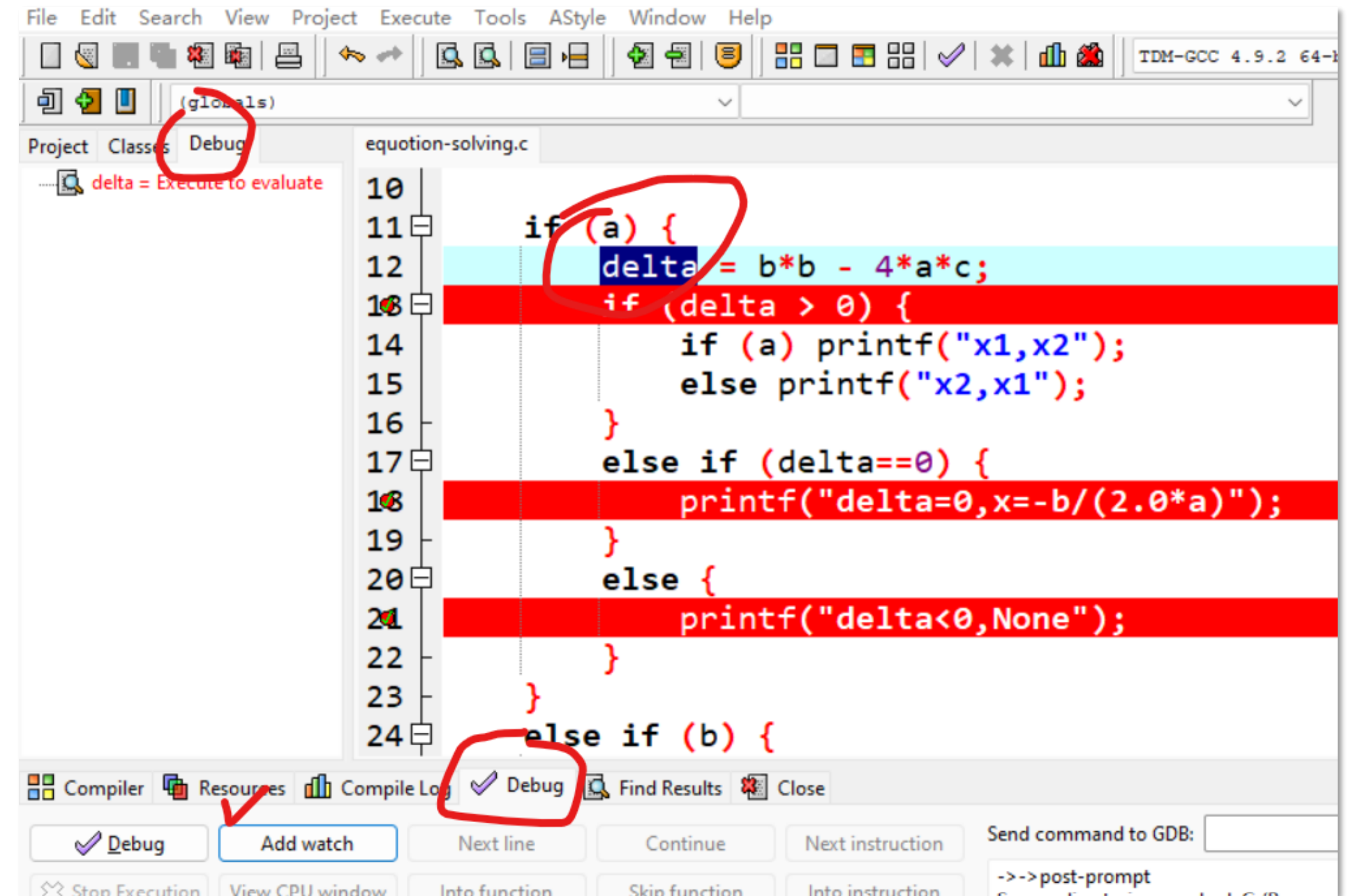
- 创建一个项目，选择 控制台应用
- 将要调试 C 代码添加到该项目
- 点击代码，打开代码
- 修改代码
- 按 F12 编译通过



调试步骤

□ 调试步骤：

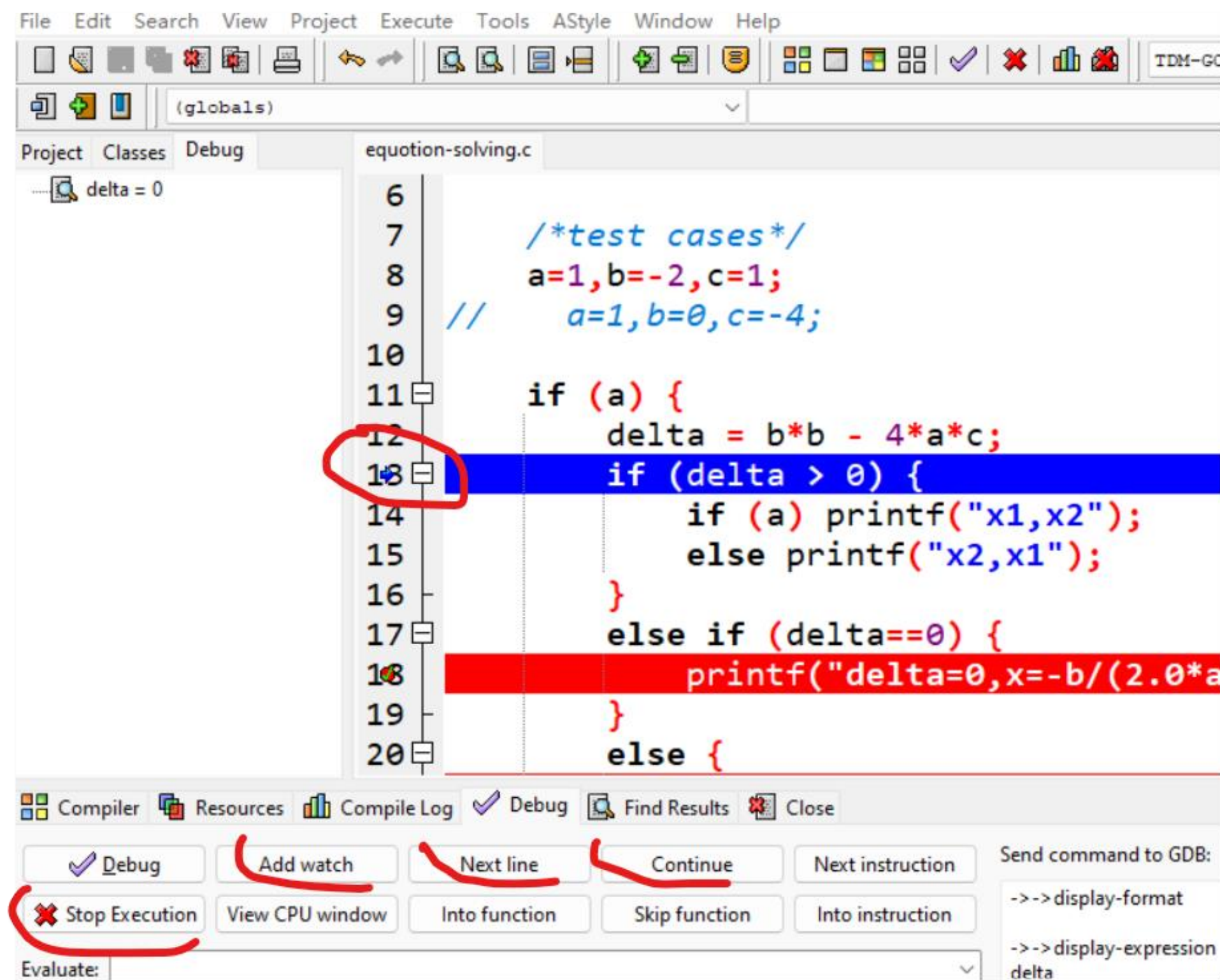
- 设置断点：点击 13、18、21 行行号，这些行变红
- 查看变量：选中变量 delta，按按钮 Add watch
- 按 F5



调试步骤

□ 调试步骤:

- 这时程序会停在第一个断点（蓝色行）
- 选中任何变量，点 Add watch 观察变量值
- 点击行号，添加/去除断点
- Next line 执行下一条语句
- Continue 执行到下一个断点或程序结束
- Stop Execution 调试结束



调试练习

❑ 断点选择和 Watch 变量

- scanf 语句的下一句，查看输入变量值
- 循环调试断点一般选择3处：
 - while, for 关键字所在行,
 - 循环体的第一行
 - 循环语句的下一行（已跳出循环）
- Watch 是 条件表达式、关键变量
- 使用 Next Line 跟踪一次迭代
- 使用 Continue 跟踪每个迭代关键变量的变化，观察迭代是否趋向结束（收敛）

总结

while(表达式)
语句

for(表达式1; 表达式2; 表达式3)
语句

do
语句
while(表达式);

break;

continue;

作业

- 阅读《C程序设计》第5章
- 完成 PTA 本周习题

谢谢